

A hand is holding a white ceramic mug filled with dark coffee. The mug is placed on a dark wooden table. In the background, there is a potted plant with long green leaves and an open book with white pages. The scene is softly lit, creating a calm and focused atmosphere.

Bom dia!
Dispostos para iniciar a aula?

Iniciaremos as 8h10min!

Vamos
alongar!





Especificação de Requisitos de Software

(Engenharia de Requisitos)

Aula 02 – Fundamentos do software

Centro de Informática – UFPB

Danielle Rousy - danielle@ci.ufpb.br

Roteiro

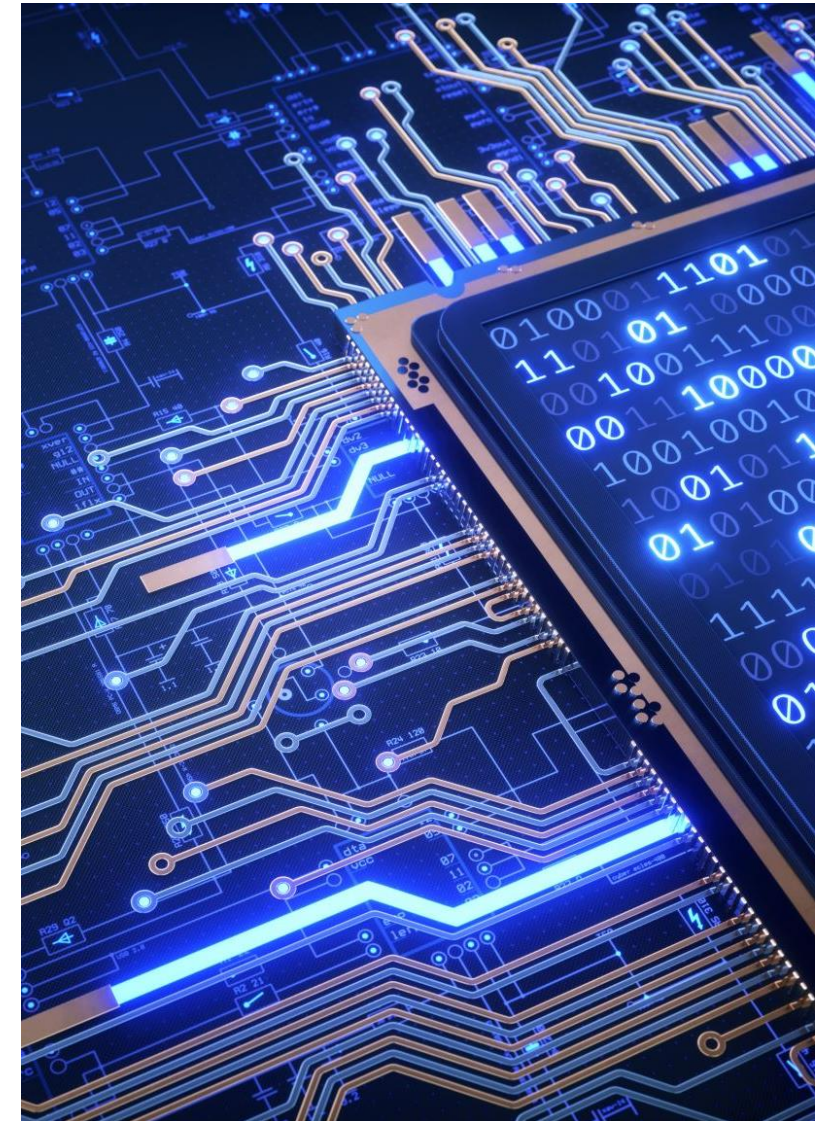
Aspectos motivacionais

Definições iniciais sobre requisitos

Falhas famosas

Engenharia de software

Engenharia de requisitos



Especificação de Requisitos de Software



“ PENSADOR

- Quando eu uso uma palavra - disse Humpty Dumpty num tom escarninho - ela significa exatamente aquilo que eu quero que signifique ... nem mais nem menos.
- A questão - ponderou Alice - é saber se o senhor pode fazer as palavras dizerem coisas diferentes.
- A questão - replicou Humpty Dumpty...

Alice no país das maravilhas Lewis Carroll

Requisitos

Software

Requisitos?

Significado de Requisito

substantivo masculino

Quesito; condição básica e necessária para se obter alguma coisa ou para alcançar determinado propósito: cumpria os requisitos para o cargo pretendido.

[Jurídico] Condição; exigência legal necessária que valida, dá validade, a uma ação jurídica.

adjetivo

Pouco Usual. Requisitado; aquilo que se solicitou; que foi requerido.

Etimologia (origem da palavra *requisito*). Do latim requisitus.a.um.

Sinônimos de Requisito

Requisito é sinônimo de: [exigência](#), [condição](#), [quesito](#)

Definição de Requisito

Classe gramatical: **adjetivo** e **substantivo masculino**

Flexão do verbo [requisitar](#) na: 1ª pessoa do singular

Separação silábica: **re-qui-si-to**

Plural: [requisitos](#)

Feminino: [requisita](#)

REQUISITOS SOFTWARE

ESTA APLICAÇÃO ESTÁ BOA MAS...
EU GOSTAVA DE VER ESTES
GRÁFICOS COM OCÚLOS 3D E
NUMA PERSPECTIVA MULTI-OMIOCU



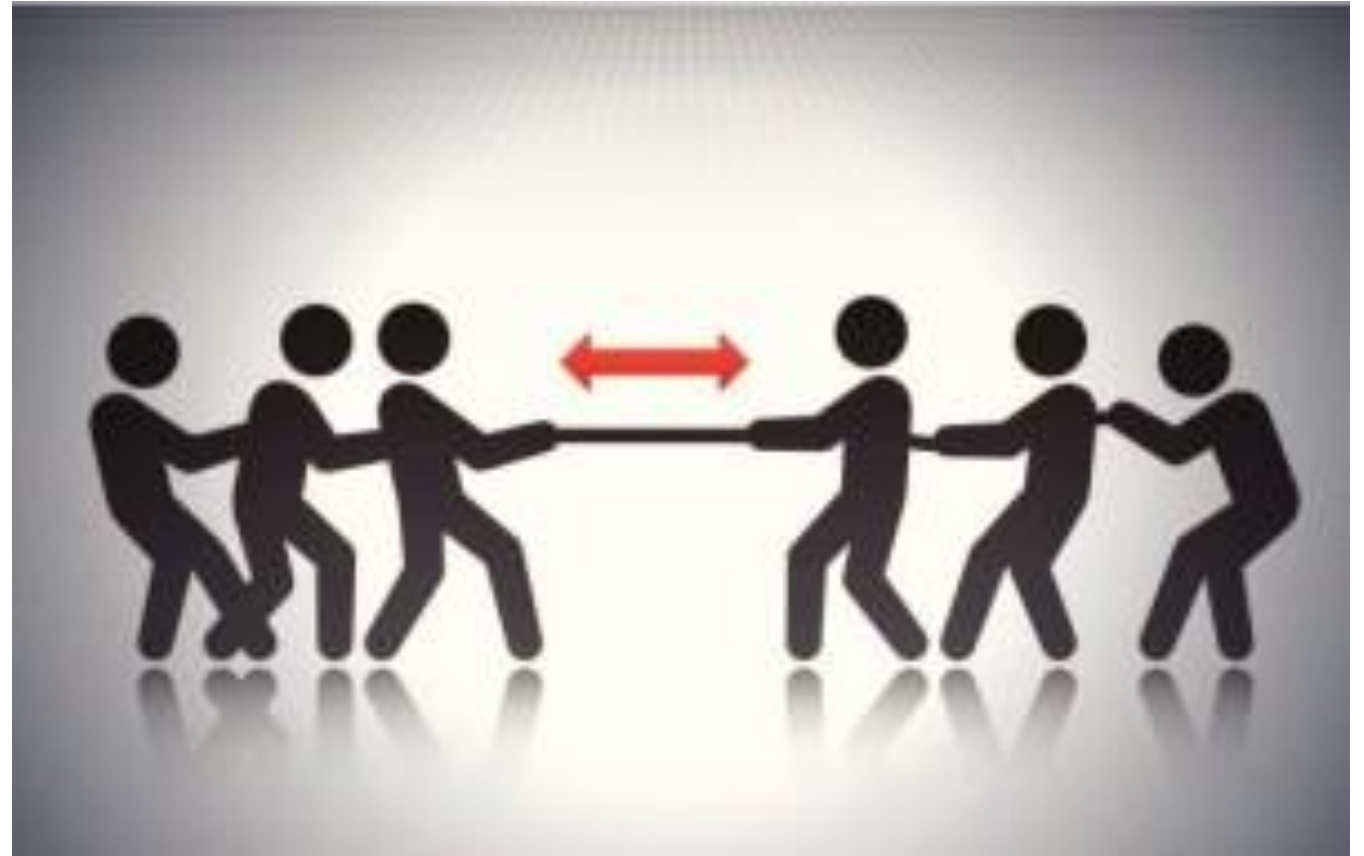
Quais os
requisitos
de uma
cadeira?



Quais os requisitos de uma cadeira?



O que faz
uma cadeira
ser melhor
do que a
outra?



O tempo
TEM UMA FORMA
MARAVILHOSA
de nos mostrar
o que realmente
IMPORTA.

“O tempo deixa
perguntas, mostra
respostas, esclarece
dúvidas, mas, acima
de tudo, o tempo
traz verdades.”



E quando
falamos de
Software?



Software?

rw.dicio.com.br/software/

Significado de Software

substantivo masculino

Programa; reunião dos procedimentos e/ou instruções que determinam o funcionamento de um computador.

[Informática] Conjunto dos elementos que, num computador, compõe o sistema de processamento de dados; todo programa que se encontra armazenado no disco rígido.

Confrontar com: [hardware](#).

plural

Pronuncia-se: /sóftuer/.

Etimologia (origem da palavra *software*). Do inglês software.

Sinônimos de Software

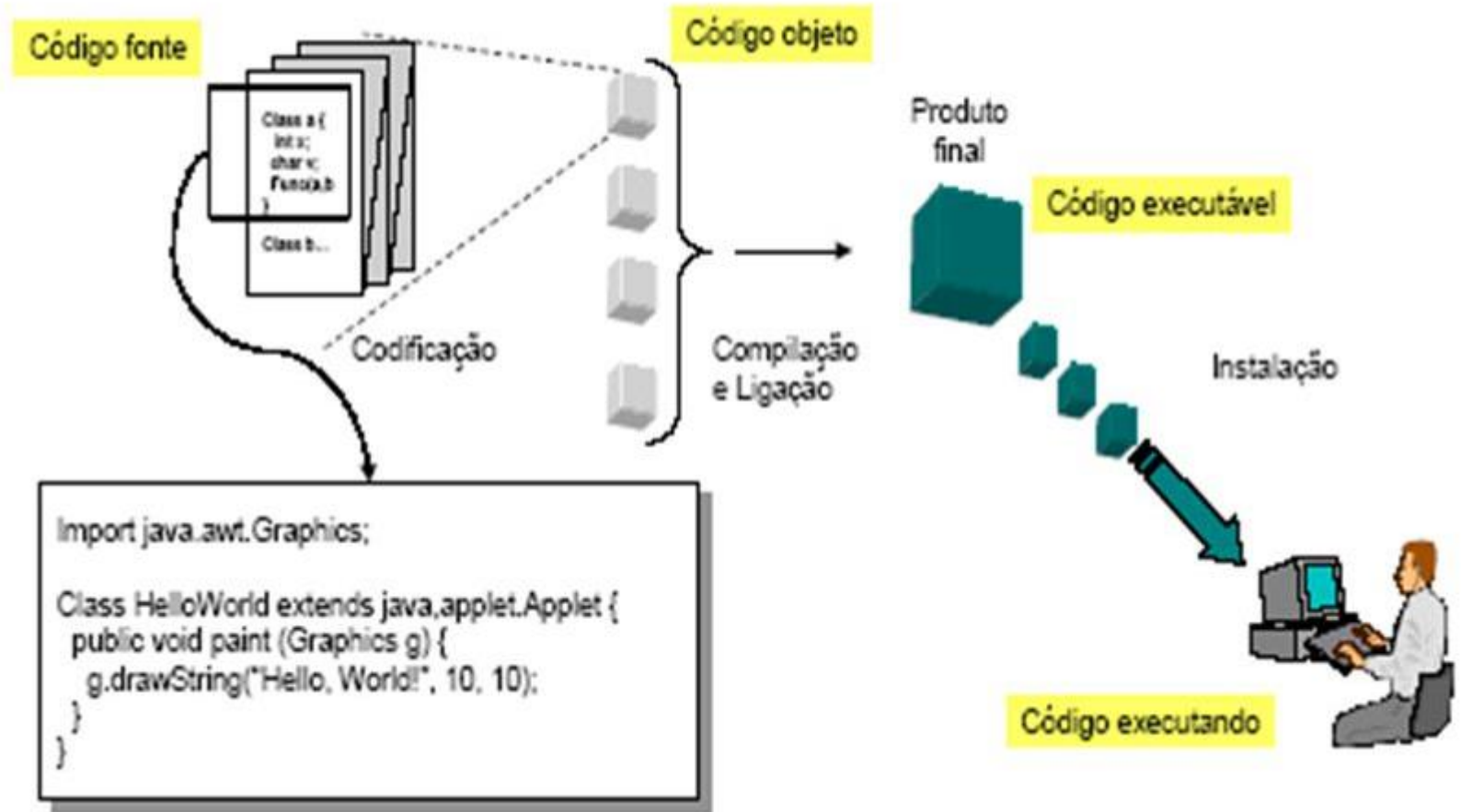
Software é sinônimo de: [programa](#)

Definição de Software

Classe gramatical: **substantivo masculino**

Separação silábica: **soft-wa-re**

Formas de software



(Pressman, 2016)

O que é um software?

- “Consiste de um conjunto de programas separados; arquivos de configuração, que são utilizados para configurar esses programas; documentação do sistema, que descreve a estrutura do sistema; a documentação do usuário, que explica como usar o sistema; e sites web por meio dos quais os usuários obtêm informações sobre o produto.” (Sommerville)

Características do software



1) É **desenvolvido** ou passa por um **processo de engenharia**; não é fabricado no sentido clássico

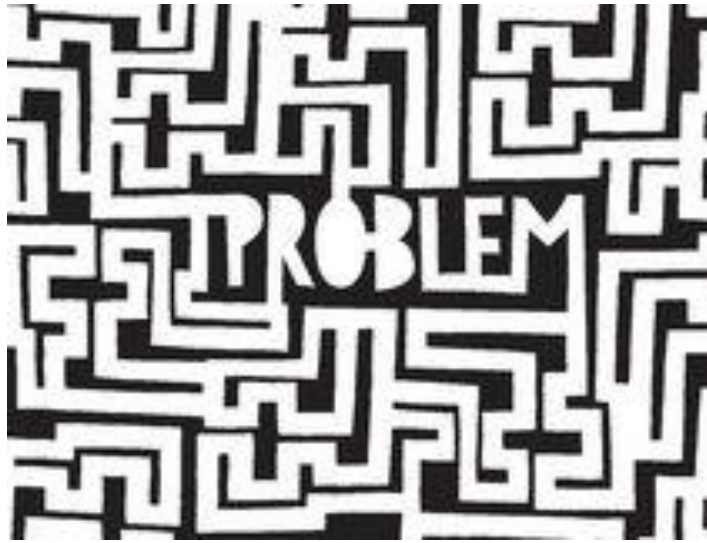


2) Não se **desgasta**



3) A maioria ainda é **feita sob medida** em vez de ser montada a partir de componentes existentes

Características do Software



Complexo



Invisível

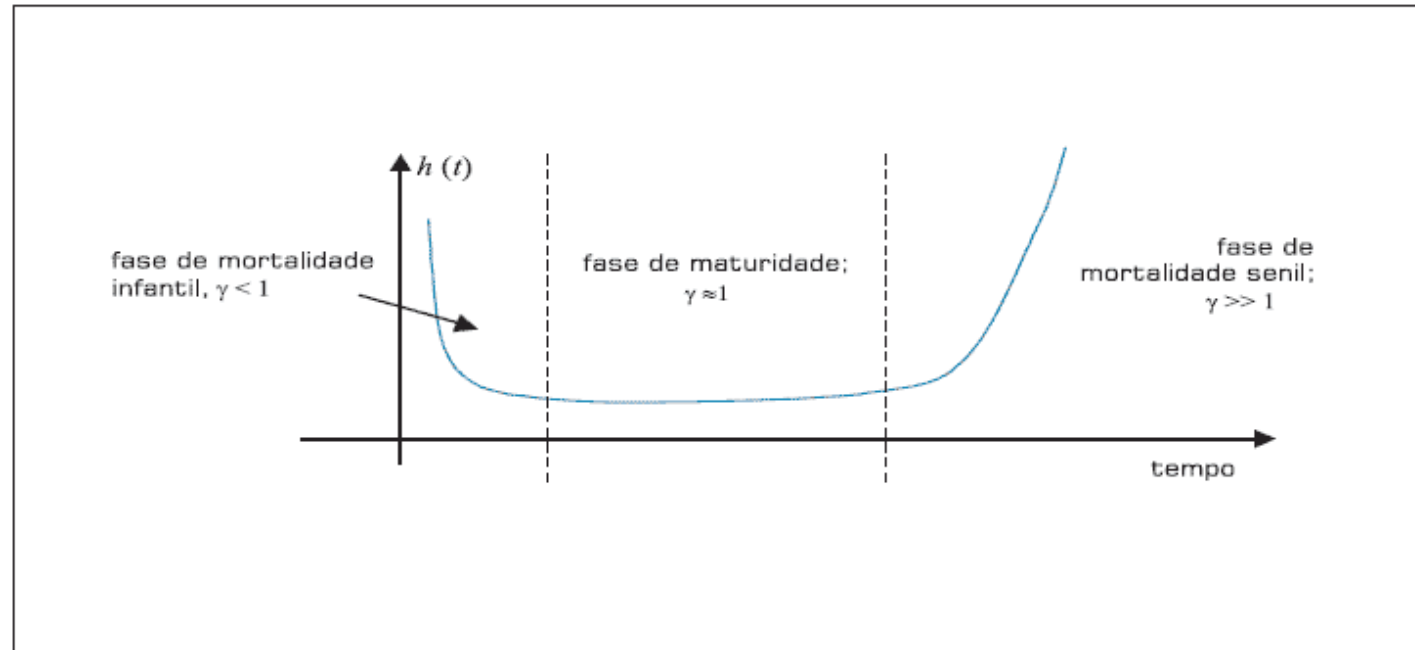


UFFD - Engenharia de Requisitos



Mutável

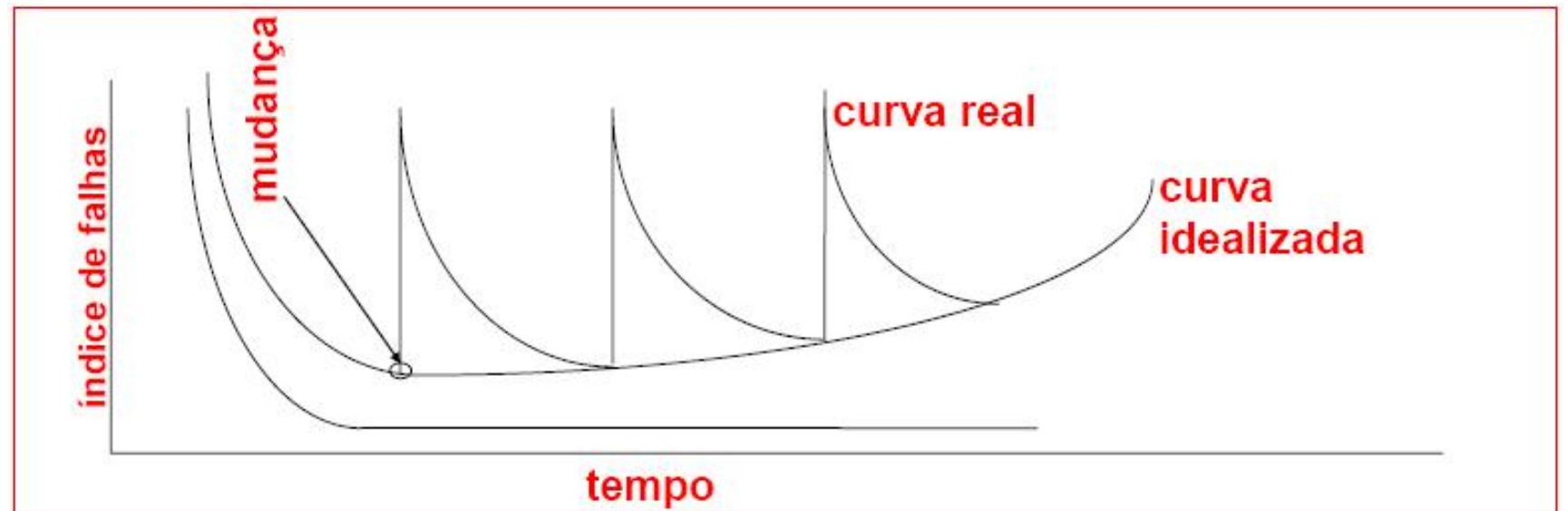
Figura 1: Curva da banheira e ciclo de vida de equipamentos.



Fonte: LAFRAIA (2001).

Hardware

Software



Curva de Falhas do Software



You are a lucky bug. I'm seeing that you'll be shipped with the next five releases.



O termo BUG...

O termo BUG



- Em 9 de Setembro de 1945, uma equipe técnica da Universidade de Harvard encontrou uma nota não usual escrita por Grace Hopper:

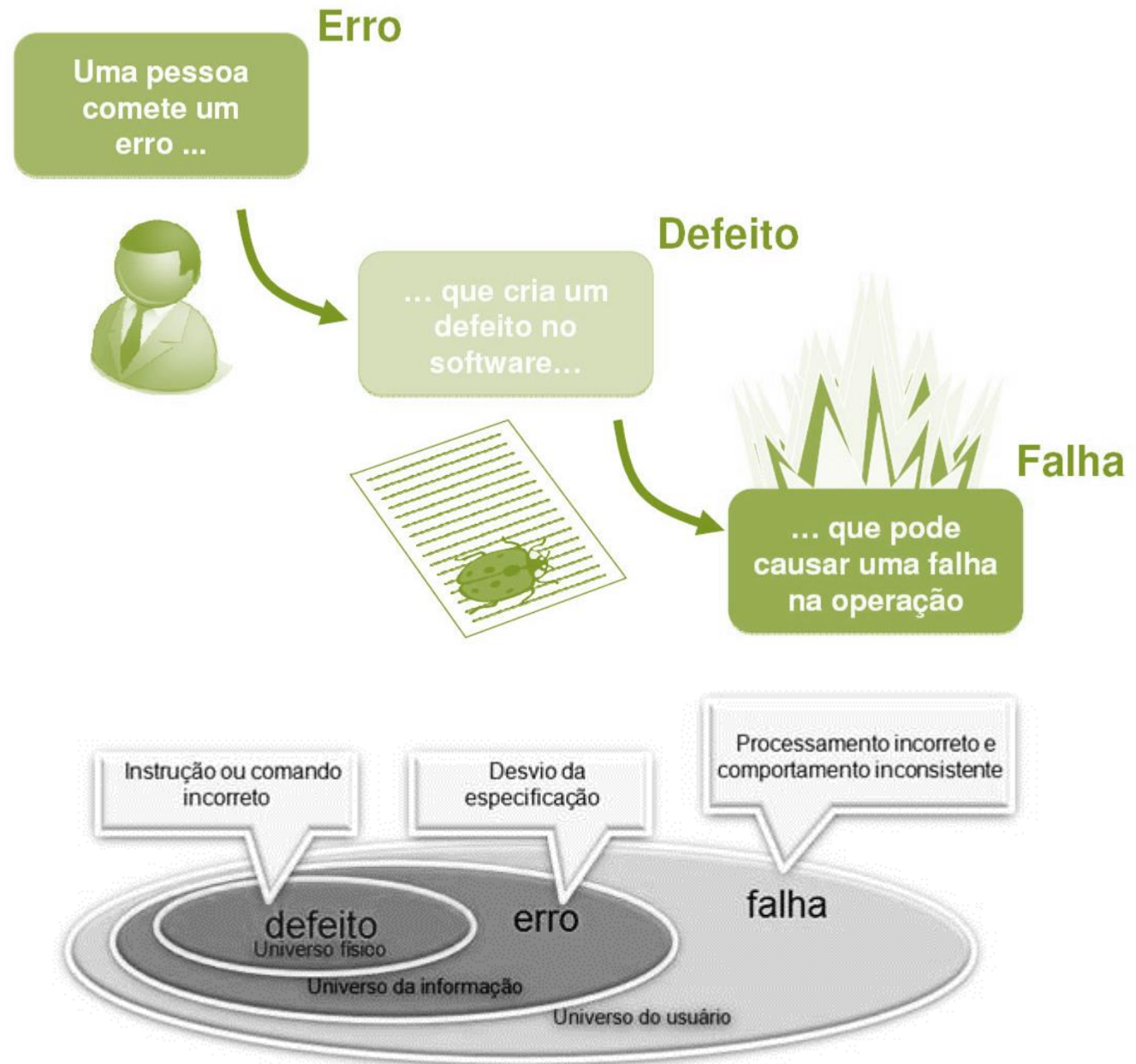
9/9

0800 Machine started
1000 stopped - machine ✓
1300 (032) MP-MC 1.48210000 9.037 847 025
033 PRO 2 2.130476415 9.037 846 995 correct
2.130476415 4.615925059(-2)
2.130476415
Relays 6-2 in 033 failed special speed test
in relay 11.00 test.
Relays changed
1100 Started Cosine Tape (Sine check)
1525 Started Multi-Adder Test.
1545 Relay #70 Panel F (moth) in relay.
First actual case of bug being found.
1630 Machine started.
1700 closed down.

Relay 2145
Relay 2370

The moth in the relay. Photo: U.S. Naval Historical Center.

Falhas? Defeitos? Erros?





Falhas
famosas de
software

FALHAS DOS FAMOSOS

Falhas famosas de software

- Ariane 5, 1996
 - O foguete explodiu 40 segundos após a sua primeira decolagem.
 - Prejuízo de U\$ 500 milhões
 - Foi aproveitado um pacote de software de navegação do Ariane 4 que não tinha erros.
 - No módulo Sistema de Referencia Inercial uma conversão de valores de 64-bits para 16-bits causou um operando inválido que o interrompeu
 - A falha no SRI levou o computador de bordo a modificar a trajetória do foguete levando à ruptura de juntas e isto causou a ativação da auto-destruição.
 - No Ariane 5, o módulo SRI só deveria ter sido utilizado antes do lançamento.

Falhas famosas de software

- London Ambulance System - despacho de ambulâncias em Londres, 1992.
 - Morte de pessoas que não foram socorridas em tempo.
 - Problema de Gerência de Software:
 - Responsáveis contrataram uma empresa desconhecida cujo valor cobrado era menor que os cobrados pelas empresas de renome.
 - Colocaram o sistema no ar sem os devidos testes.
 - Não foi feita uma migração correta do sistema antigo para o novo.

Falhas famosas de software

- Máquina de radioterapia com software controlador (1985- 87)
 - O controle de segurança feito pelo hardware em máquinas anteriores foi removido e passou a ser feito pelo software.
 - O software falhou na tarefa de, ao mesmo tempo, manter invariantes essenciais: o feixe de elétrons e o dispositivo que controla a concentração do feixe em níveis seguros.
 - Mortes ocasionadas em decorrência de queimaduras.
 - O programador não tinha experiência em programação concorrente.

Falhas famosas do software

- Sistema de triagem/control de bagagem do aeroporto internacional de Denver (EUA)
 - Atrasou a inauguração do aeroporto. Custo do sistema: US\$ 193 milhões
 - Inauguração estava prevista para Out/1993.
 - Em Junho/1994 o sistema ainda não estava funcionando e causava prejuízos de US\$ 1,1 milhão/dia
 - No começo de 1995 um controle MANUAL de bagagem foi instalado para que o aeroporto pudesse ser inaugurado (com atraso de mais de um ano)
 - Problema de planejamento e gerenciamento.

O que esses
erros tem em
comum?

QUANDO E COMO FORAM INTRODUZIDOS?

PODERIAM SER EVITADOS?



Pausa para o café!!!



Onde estão os
dois
convidados?



Encontre a
cobra?



A stylized illustration of a man with brown hair, wearing a dark suit, white shirt, and dark tie. He has a worried expression with wide eyes and a slightly open mouth. He is standing next to a large, thick red arrow that points downwards, forming a V-shape. The background is teal with horizontal dashed white lines. The text "Crise do software? O que é isso?" is overlaid on the left side of the image.

Crise do
software? O
que é isso?

Crise de software

Refere-se a um conjunto de problemas encontrados no desenvolvimento de software.



Crise de software

- **(1) As estimativas de prazo e de custo freqüentemente imprecisas**
 - “Não dedicamos tempo para coletar dados sobre o processo de desenvolvimento de software”
 - “Com poucos dados históricos como guia as estimativas tem sido a olho, com resultados previsivelmente ruins”
- **(2) A produtividade das pessoas da área de software não tem acompanhado a demanda por seus serviços**
 - “Os projetos de desenvolvimento de software normalmente são efetuados apenas com um vago indício das exigências do cliente”

Crise de software

- **(3) A qualidade de software às vezes é menos que adequada**
 - Só recentemente começam a surgir conceitos quantitativos sólidos de garantia de qualidade de software
- **(4) O software existente é muito difícil de manter**
 - A tarefa de manutenção devora o orçamento destinado ao software
 - A facilidade de manutenção não foi enfatizada como um critério importante

Algumas causas possíveis...

- **1) próprio caráter do Software**
 - O software é um elemento de sistema lógico e não físico.
 - Consequentemente, o sucesso é medido pela qualidade de uma única entidade e não pela qualidade de muitas entidades manufaturadas
- **Lembrando que o software não se desgasta, mas se deteriora!!!**

Algumas causas possíveis...

- 2) falhas das pessoas responsáveis pelo desenvolvimento de Software
 - Gerentes sem nenhum *background* em software
 - Os profissionais da área de software têm recebido pouco treinamento formal em novas técnicas para o desenvolvimento de software
 - Resistência a mudanças.
 - ...

Algumas
causas
possíveis...

- **3) mitos do software**
 - propagaram desinformação e confusão
 - administrativos
 - cliente
 - profissional

Mitos do software (Pressman)

O estabelecimento de objetivos gerais é suficiente para se começar a escrever programas.

Dê a uma pessoa técnica um bom livro de programação e você terá um programador.

Mudanças no software podem ser feitas facilmente porque ele é "flexível".

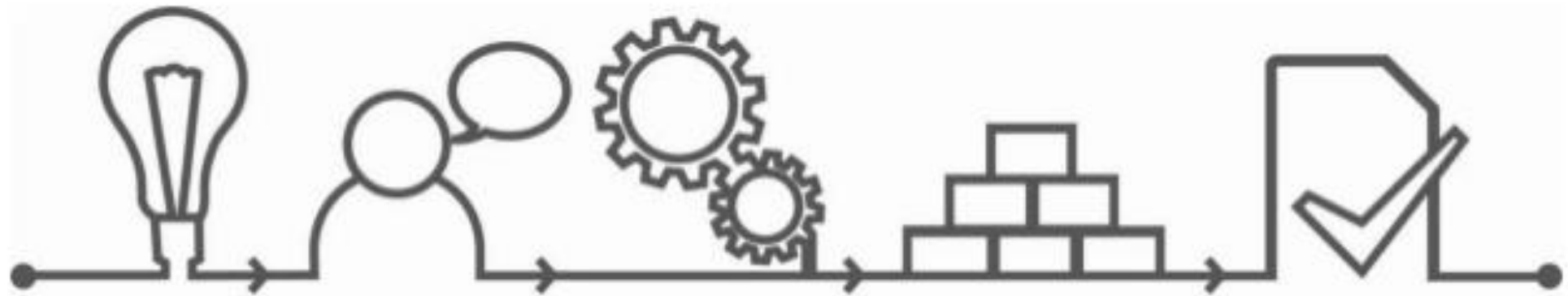
Até que o programa esteja "rodando" não é possível verificarmos a sua qualidade.

Uma vez que o programa esteja escrito e funcionando, nosso trabalho está feito.

Um projeto é bem sucedido se conseguirmos um programa funcionando corretamente.

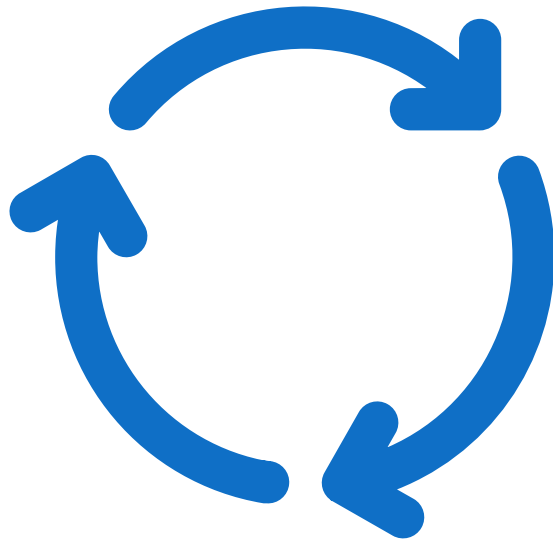


Engenharia de Software



"Engenharia de *Software* é a criação e a utilização de sólidos princípios de engenharia a fim de obter software de maneira econômica, que seja confiável e que trabalhe em máquinas reais"

BAUER, 1960



Engenharia de Software

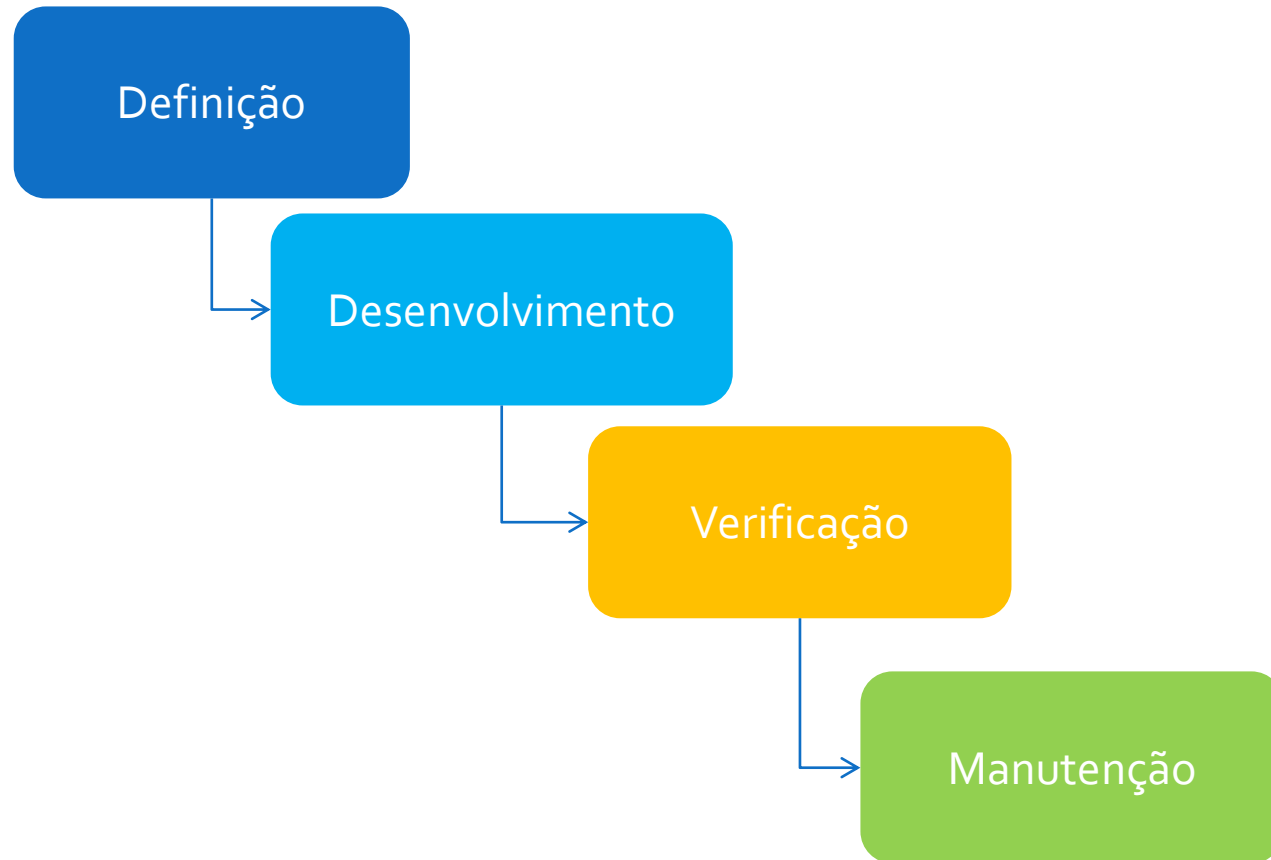
“Disciplina da engenharia relacionada com todos os aspectos da produção de um software, desde os estágios iniciais da especificação de sistemas até sua manutenção, depois que este entrar em operação”. (Sommerville)



Processo de Software

Representa um conjunto de ATIVIDADES, MÉTODOS, FERRAMENTAS e resultados associados que produz um PRODUTO DE SOFTWARE.

Paradigma genérico de processo



Fase de Definição

análise de sistema
planejamento do projeto de software
análise de requisitos

Fase de Desenvolvimento

projeto de software
codificação

Fase de Verificação

teste de software

Fase de Manutenção

adaptação
correção
perfectiva

Métodos

- Proporcionam os detalhes de como fazer para construir o software
 - Planejamento e estimativa de projeto
 - Análise de requisitos e de sistemas
 - Projeto de estrutura de dados

Ferramentas

- Dão suporte automatizado aos métodos
- Existem ferramentas para sustentar cada um dos métodos
- Quando as ferramentas são integradas é estabelecido um sistema de suporte ao desenvolvimento de software chamado CASE (*Computer Aided Software Engineering*)

Como tudo começa...

- Todo projeto de software é iniciado por uma necessidade do negócio – a necessidade de corrigir um defeito em uma aplicação existente; a necessidade de adaptar um sistema legado para mudar o ambiente de negócio; a necessidade de estender as funções e características de uma aplicação existente; ou a necessidade de criar um novo produto, serviço ou sistema

Qual a **necessidade de negócio ou problema** que você vai trabalhar?

Engenharia de Requisitos

Para assegurar que uma solução de software resolve um problema particular, devemos entender e definir que problema necessita ser resolvido

Descobrir, entender, formular e analisar

Qual? Por que? Quem?

E isto é o que a Engenharia de Requisitos faz





CHAOS REPORT



**THE
STANDISH
GROUP**



	1994	1995	2000	2001	2002	2003
Plant Assets	14.5	100%	148.0	174.0	49.0	200%
Goodwill	0.00	0.00	49.0	49.0	49.0	49.0
Total	14.5	100%	197.0	223.0	98.0	249%



Chaos Report, 2014

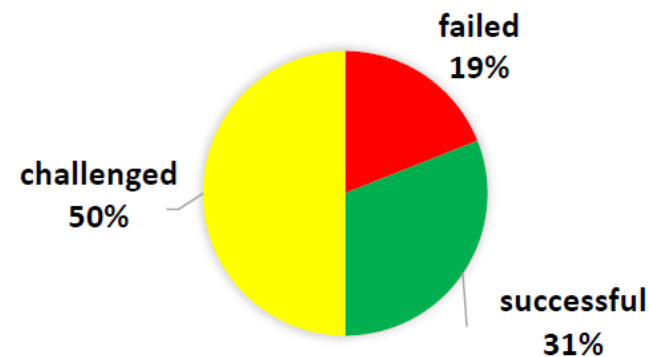
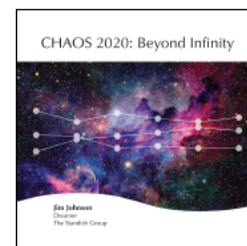
Project Success Factors	% of Responses
1. User Involvement	15.9%
2. Executive Management Support	13.9%
3. Clear Statement of Requirements	13.0%
4. Proper Planning	9.6%
5. Realistic Expectations	8.2%
6. Smaller Project Milestones	7.7%
7. Competent Staff	7.2%
8. Ownership	5.3%
9. Clear Vision & Objectives	2.9%
10. Hard-Working, Focused Staff	2.4%
Other	13.9%

Chaos Report, 2014

Project Challenged Factors	% of Responses
1. Lack of User Input	12.8%
2. Incomplete Requirements & Specifications	12.3%
3. Changing Requirements & Specifications	11.8%
4. Lack of Executive Support	7.5%
5. Technology Incompetence	7.0%
6. Lack of Resources	6.4%
7. Unrealistic Expectations	5.9%
8. Unclear Objectives	5.3%
9. Unrealistic Time Frames	4.3%
10. New Technology	3.7%
Other	23.0%

Project Success Quick Reference Card

Based on CHAOS 2020: Beyond Infinity Overview. January 2021, QRC by Henny Portman



Modern measurement (software projects)

Good Sponsor, Good Team, and Good Place are the only things we need to improve and build on to improve project performance.



The Good Place is where the sponsor and team work to create the product. It's made up of the people who support both sponsor and team. These people can be helpful or destructive. It's imperative that the organization work to improve their skills if a project is to succeed. This area is the hardest to mitigate, since each project is touched by so many people. Principles for a Good Place are:

- The Decision Latency Principle
- The Emotional Maturity Principle
- The Communication Principle
- The User Involvement Principle
- The Five Deadly Sins Principle
- The Negotiation Principle
- The Competency Principle
- The Optimization Principle
- The Rapid Execution Principle
- The Enterprise Architecture Principle



Successful project Resolution by Good Place Maturity Level:

highly mature	50%
mature	34%
moderately mature	23%
not mature	23%

The Good Team is the project's workhorse. They do the heavy lifting. The sponsor breathes life into the project, but the team takes that breath and uses it to create a viable product that the organization can use and from which it derives value. Since we recommend small teams, this is the second easiest area to improve. Principles for a Good Team are:

- The Influential Principle
- The Mindfulness Principle
- The Five Deadly Sins Principle
- The Problem-Solver Principle
- The Communication Principle
- The Acceptance Principle
- The Respectfulness Principle
- The Confrontationist Principle
- The Civility Principle
- The Driven Principle



Successful project Resolution by Good Team Maturity Level:

highly mature	66%
mature	46%
moderately mature	21%
not mature	1%

The Good Sponsor is the soul of the project. The sponsor breathes life into a project, and without the sponsor there is no project. Improving the skills of the project sponsor is the number-one factor of success – and also the easiest to improve upon, since each project has only one.

Principles for a Good Sponsor are:

- The Decision Latency principle
- The Vision Principle
- The Work Smart Principle
- The Daydream Principle
- The Influence Principle
- The Passionate Principle
- The People Principle
- The Tension Principle
- The Torque Principle
- The Progress Principle



Successful project Resolution by Good Sponsor Maturity Level:

highly mature	67%
mature	33%
moderately mature	21%
not mature	18%

Fonte: Naming the Pain
in Requirements Engineering:
Contemporary Problems,
Causes, and Effects in Practice
(<https://arxiv.org/abs/1611.10288>)

- Requisitos incompletos ou não-documentados (48%)
- Falhas de comunicação entre membros do time e os clientes (41%)
- Requisitos em constante mudança (33%)
- Requisitos especificados de forma abstrata (33%)
- Restrições de tempo (32%)
- Problemas de comunicação entre os próprios membros do time (27%)
- Stakeholders com dificuldades de separar requisitos e soluções (25%)
- Falta de apoio dos clientes (20%)
- Requisitos inconsistentes (19%)
- Falta de acesso às necessidades dos clientes ou do negócio (18%)

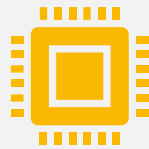
Requisitos de software

- Requisitos são, além de funções, objetivos, propriedades, restrições que o sistema deve possuir para satisfazer contratos, padrões ou especificações de acordo com o(s) usuário(s).
- É uma condição necessária para satisfazer um objetivo.

Engenharia de Requisitos

- Disciplina da **Engenharia de Software**
- Consiste no uso sistemático e repetitivo de técnicas para cobrir atividades de obtenção, documentação e manutenção de um conjunto de requisitos para software que atendam aos objetivos de negócio e sejam de qualidade

Envolve,
muitas vezes
entender 2
versões do
mesmo
sistema...



A primeira que explica o sistema antes do software ser construído



A segunda que explica o sistema com o software a ser construído e operacionalizado

Por exemplo....



Sistema de Alocação de Salas para o CI



LEITURA

OBRIGATÓRIA

Leitura Obrigatória

- PRESSMAN, Roger, MAXIM, Bruce. (01/2016). *Engenharia de Software, 8th edição*. [Minha Biblioteca]. Retirado de <https://integrada.minhabiblioteca.com.br/#/books/9788580555349/>
- Capítulo 01 – A natureza do software
- Capítulo 02 – Engenharia de software



Fazer resumo
em duas folhas
sobre os
capítulos!

Pontos discutidos...

- O software tem propriedades intrínsecas que o diferencia de outros produtos, como o hardware, e impactam em seu ciclo de desenvolvimento
- Software é uma entidade lógica, conceitual, invisível, complexa, mutável e criado em conformidade
- É importante que o ciclo de desenvolvimento do software seja definido de forma a proporcionar um produto de melhor qualidade reduzindo as dificuldades de controle do progresso e manutenção
- Foi nesse sentido que a engenharia de software surgiu - no intuito de melhorar a produção do software
- E todo o processo de engenharia de software, inicia-se com a Engenharia de requisitos – entendimento, descoberta, especificação e controle dos requisitos do produto

Próxima aula..

- Continuaremos falando sobre os fundamentos, direcionado aos processos de desenvolvimento...